Queen Mary
University of London

EBU5376
Digital Signal Processing

Fast Fourier Transform (FFT) – Part 1

Module Organizer: Dr Yixuan Zou yixuan.zou@qmul.ac.uk

Outline

- From DFT to FFT
  - Computational Complexity
  - Divide and Conquer
  - Key Properties of DFT
- Bit-reversal

Is DFT Good Enough?

$$\boxed{X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}}$$

- For each frequency k we choose , we must multiply each signal value $x[n]$ by a complex number W_N^{kn} and add together the results.
- For each frequency component, we have N complex multiplications and N-1 complex additions to perform. Consequently, each frequency requires

$$N + (N - 1) = 2N - 1$$

basic computational steps. As we have N frequencies, the total number of computations is

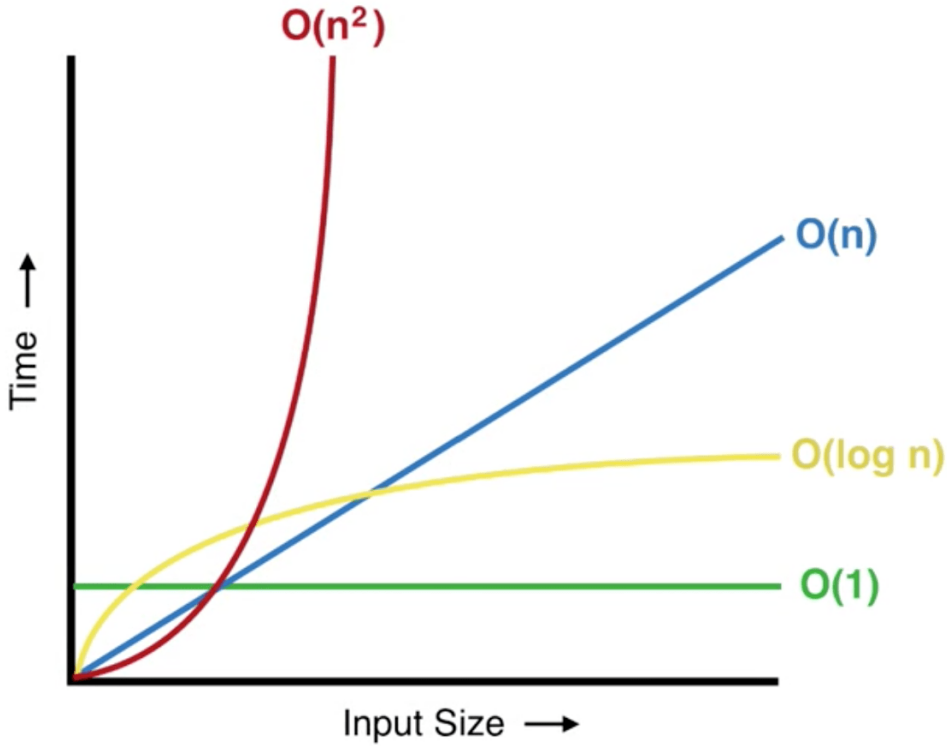
$$N(2N - 1) = 2N^2 - N$$

Is DFT Good Enough?

- The computational complexity of DFT is $2N^2 - N$
- N is the length of the signal
- For example:
  - For 10 samples, DFT needs to perform 190 operations.
  - For 100 samples, DFT needs to perform 19900 operations.
  - Adding 90 more samples results in 19710 more operations!

Big O Notation for Complexity

- Big O notation, often denoted as $O(\cdot)$, is a mathematical notation used in computer science to describe the behaviour of an algorithm's time complexity.
- In a N-point DFT, we need to perform a total of $2N^2 - N$ complex calculations. The algorithm complexity of DFT is $O(N^2)$



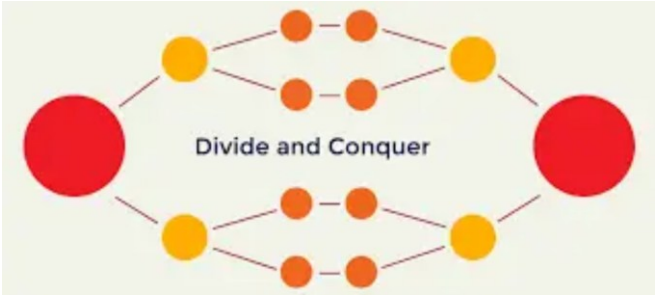
Is DFT Good Enough?

- Consider N = 10^9 and assume 10^{-9} second per operation
  - $\mathcal{O}(N^2)$ =10^{18} operations ≈ 3 years ➡ DFT
  - $\mathcal{O}(NlogN)$ = $9 * 10^9$ operations ≈ 9 seconds

⬇

Fast Fourier Transform (FFT)

FFT – Divide and Conquer



- The divide-and-conquer approach is a fundamental concept used in many algorithms, including FFT

- In the context of FFT, the divide-and-conquer approach is used to break down the computation of the FFT into smaller, simpler sub-problems that can be solved recursively.

From DFT to FFT

- Before 1965, nearly all DFTs were calculated using $O(N^2)$ algorithms
- FFT is designed based on various properties of DFT:

$$W_N^{k+N} = W_N^k$$

$$W_N^{\frac{N}{2}+k} = -W_N^k$$

$$W_N^{(2n+1)k} = W_{N/2}^{nk} W_N^k$$

- We will study each of them and explain how they enable the FFT algorithm

From DFT to FFT

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, W_N^{kn} = e^{-\frac{j2\pi kn}{N}}$$

- Periodicity of DFT

$$W_N^{k+N} = W_N^k$$

Proof:

$$\begin{aligned}
\text{LHS} = W_N^{k+N} &= e^{-j2\pi\frac{k+N}{N}} = e^{-j2\pi\frac{k}{N}} e^{-j2\pi\frac{N}{N}} \\
&= e^{-j2\pi\frac{k}{N}} e^{-j2\pi} \\
&= e^{-j2\pi\frac{k}{N}} \cdot (\cos(-2\pi) + j\sin(-2\pi)) \\
&= e^{-j2\pi\frac{k}{N}} (1) \\
&= e^{-j2\pi\frac{k}{N}} = W_N^k = \text{RHS}
\end{aligned}$$

From DFT to FFT

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, W_N^{kn} = e^{-\frac{j2\pi kn}{N}}$$

- Periodicity of DFT

  - Example: In 8-point DFT, the values of W_N^{kn} for $k = 1$ and $k = 5$ are

$\underline{k = 5:}$ $\{W_8^{5n}, n = 0, \dots, 7\} = W_8^0, W_8^5, W_8^{10}, W_8^{15}, W_8^{20}, W_8^{25}, W_8^{30}, W_8^{35}$

$= W_8^0, W_8^5, W_8^2, W_8^7, W_8^4, W_8^1, W_8^6, W_8^3$

$\underline{k = 1:}$ $\{W_8^n, n = 0, \dots, 7\} = W_8^0, W_8^1, W_8^2, W_8^3, W_8^4, W_8^5, W_8^6, W_8^7$

Same 8 coefficients!

From DFT to FFT

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, W_N^{kn} = e^{-\frac{j2\pi kn}{N}}$$

- Half-N Periodicity of DFT

$$W_N^{N/2} = -1 \quad \Rightarrow \quad W_N^{\frac{N}{2}+k} = -W_N^k$$

- Since twiddle factors exhibit this structured repetition, we only need to compute the first $N/2$ **twiddle factors** explicitly, and the remaining ones can be derived efficiently.
- For example:

$$\{W_8^n, n = 0, \dots, 7\} = W_8^0, W_8^1, W_8^2, W_8^3, W_8^4, W_8^5, W_8^6, W_8^7$$
$$= W_8^0, W_8^1, W_8^2, W_8^3, -W_8^0, -W_8^1, -W_8^2, -W_8^3$$

From DFT to FFT

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, W_N^{kn} = e^{-\frac{j2\pi kn}{N}}$$

- Periodicity of DFT
  - Without utilizing the periodicity, we re-evaluate W_N^{kn} when calculating every $X[k]$. It leads to lot of duplicated calculations.
  - By identifying the periodicity, we only need to evaluate W_N^{kn} for N times and reuse the result in every $X[k]$.
  - However, periodicity can only reduce the complexity by a constant factor
  - The overall complexity is still $O(N^2)$

From DFT to FFT

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, W_N^{kn} = e^{-\frac{j2\pi kn}{N}}$$

- Decomposition of DFT

$$W_{Nm}^{nkm} = W_{N/m}^{nk/m} = W_N^{nk} = e^{-j\frac{2\pi nk}{N}}$$

Based on the above relationships, we can also derive:

$$W_N^{2nk} = W_{N/2}^{nk}$$

$$W_N^{(2n+1)k} = W_{N/2}^{nk} W_N^k$$

From DFT to FFT

- Below is the mathematical derivation that splits a N-point DFT into 2 N/2-point DFTs

Split into sums of even-indexed $x[2n]$ and odd-indexed $x[2n+1]$

$$
\begin{aligned}
X(k) &= \sum_{n=0}^{N-1} x[n] W_N^{nk} \\
&= \sum_{n=0}^{N/2-1} x[2n] W_N^{2nk} + \sum_{n=0}^{N/2-1} x[2n+1] W_N^{(2n+1)k} \\
&= \sum_{n=0}^{N/2-1} x_1[n] W_{N/2}^{nk} + W_N^k \sum_{n=0}^{N/2-1} x_2[n] W_{N/2}^{nk} \\
&= X_1(k) + W_N^k X_2(k)
\end{aligned}
$$

N/2-point DFT

N/2-point DFT

Denote:

Even-indexed inputs

$x_1[n] = x[2n]$

Odd-indexed inputs

$x_2[n] = x[2n+1]$

$$W_N^{2nk} = W_{N/2}^{nk}$$

$$W_N^{(2n+1)k} = W_{N/2}^{nk} W_N^k$$

From DFT to FFT

- Now we have obtained the following decomposition:

$$X(k) = X_1(k) + W_N^k X_2(k) \qquad k = 0,1,\dots,N-1$$

Note that

- The two N/2 DFTs $X_1[k]$ and $X_2[k]$ are periodic with period N/2 (e.g.,$X_1\left[k+\frac{N}{2}\right] = X_1[k]$)
- Hence, by further applying the half-N periodicity property $\left(W_N^{\frac{N}{2}+k} = -W_N^k\right)$, the above equation can be expressed as:

$$X(k) = X_1(k) + W_N^k X_2(k) \qquad k = 0,1,\dots,N/2-1$$

$$X(k+N/2) = X_1(k+N/2) + W_N^{k+N/2} X_2(k+N/2)$$
$$= X_1(k) - W_N^k X_2(k) \qquad k = 0,1,\dots,N/2-1$$

From DFT to FFT

$$\begin{aligned} X(k) &= X_1(k) + W_N^k X_2(k) \\ X(k+N/2) &= X_1(k) - W_N^k X_2(k) \end{aligned} \qquad k = 0,1,\ldots,N/2-1$$

- For example, when $N = 4$, the decomposition equations becomes

From DFT to FFT

- We have shown that an N-point DFT can be decomposed into two N/2-point DFTs
- We can further divide the N/2-point DFTs into N/4-point DFT, and so on…
- Until we reach a batch of 2-point DFTs, which cannot be further divided
- Hence, any N-point DFT can be decomposed into a summation of several **2-point DFTs**
- Particularly, an N-point DFT can only be decomposed for $\log_2(N)$ times
- To compute FFT, we first need to rearrange the input sequence to match the recursive decomposition structure. This leads us to the concept of **bit-reversal ordering**

Outline

- From DFT to FFT
  - Computational Complexity
  - Divide and Conquer
  - Key Properties of DFT
- Bit-reversal

Bit Reversal

- Bit reversal involves rearranging the order of the input sequence based on the binary representation of their indices

- Recall: In FFT, N-point DFT is decomposed into several 2-point DFTs. The consecutive pairs in the bit-reversed input are the inputs to these 2-point DFTs.

- Binary representation is a way of representing numbers in the base-2 numbering system, where only two digits, 0 and 1, are used.

Bit Reversal

➢ Binary representation

- An N-bit binary representation indicates that each number is expressed as a length-N sequence of 0s and 1s.
- The binary representation of number M requires at least $N = ceil(\log_2 M)$ number of bits
- For example:
  - The binary representation of $M = 9$ requires at least $N = ceil(\log_2 9) = ceil(3.16) = 4$ bits.
  - Using 8 bits to represent a number allows for a range of values from 0 to 255, while using 16 bits allows for a range from 0 to 65,535.

Bit Reversal

➢ Binary representation

❖ Steps to calculate the binary representation of M:

1. Decompose $M = 2q_1 + r_1, r_1 \in \{0,1\}$
2. Decompose $q_1 = 2q_2 + r_2$
3. Repeat step 2 for $q_2, q_3, \ldots$ until $q_k = 0$ for some k
4. The binary representation for M is $r_k \ldots r_2 r_1$

Bit Reversal

- **Binary representation**
  - Example: The binary representation of 6 can be obtained by
    1. Divide 6 by 2: $q_1 = 3$, $r_1 = 0$.
    2. Divide 3 by 2: $q_2 = 1$, $r_2 = 1$.
    3. Divide 1 by 2: $q_3 = 0$, $r_3 = 1$.

$\Big\}$ 110

Decimal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Bit Reversal

- After obtaining the binary representation, bit reversal is achieved by reversing the binary representation, i.e.,
  - 001 →(reversed) 100
  - 011 →(reversed) 110

- Steps to perform bit reversal
  1. Map numbers to binary representations
  2. Reverse the binary representations
  3. Map from binary to numbers

- In FFT, the 'number' is the index of the input, i.e., the index of $x[n]$ is n

Bit Reversal

- Example:

0
1
2
3
4
5
6
7

convert to binary →

reverse the bits →

map to number →

Bit Reversal in FFT

- In FFT, bit reversal involves reordering the input data in a specific way to optimize the computational efficiency of the algorithm.
- For example, if we have an input array with 8 elements $[x[0], x[1], x[2], x[3], x[4], x[5], x[6], x[7]]$, the bit-reversed input would be $[x[0], x[4], x[2], x[6], x[1], x[5], x[3], x[7]]$.
- If we pair up each consecutive inputs, we get

  $$\{x[0], x[4]\}, \{x[2], x[6]\}, \{x[1], x[5]\}, \{x[3], x[7]\}$$

  look familiar?
- *Trick*: By repeatedly dividing the inputs according to odd-even indices, we get the bit-reversed inputs. Only useful for small number of inputs.

Summary

- An N-point DFT can be decomposed into two N/2-point DFT, as shown below:

$$X(k) = X_1(k) + W_N^k X_2(k)$$

$$X(k + N/2) = X_1(k) - W_N^k X_2(k)$$

$$k = 0,1, \dots, N/2 - 1$$

Here $X[k]$ is the N-point DFT, $X_1[k]$ and $X_2[k]$ are the N/2-point DFTs

- More particularly, $X_1[k]$ is the DFT of the even-indexed inputs, and $X_2[k]$ is the DFT of the odd-indexed inputs.
- This decomposition can be done recursively, until we reach a set of 2-point DFTs, which form the fundamental computation units in the FFT algorithm

Summary

- FFT decomposition splits inputs into even- and odd-indexed terms, causing indices to appear out of order.
- **Bit-reversal** reorders the inputs so that the indices follow the correct sequence for recursive computation
- Thus, the FFT algorithm follows these steps:
  1. Pair up consecutive bit-reversed inputs
  2. Compute the 2-point DFT of each pair (Producing N outputs)
  3. Pair up the new outputs and compute the 2-point DFT of each pair
  4. Repeat this process for a total of $\log_2 N$ stages
  5. We reach N final outputs: $X[0], \dots, X[N-1]$

Queen Mary
University of London

MATLAB - FFT

- FFT in MATLAB can be done with a simple function (link):

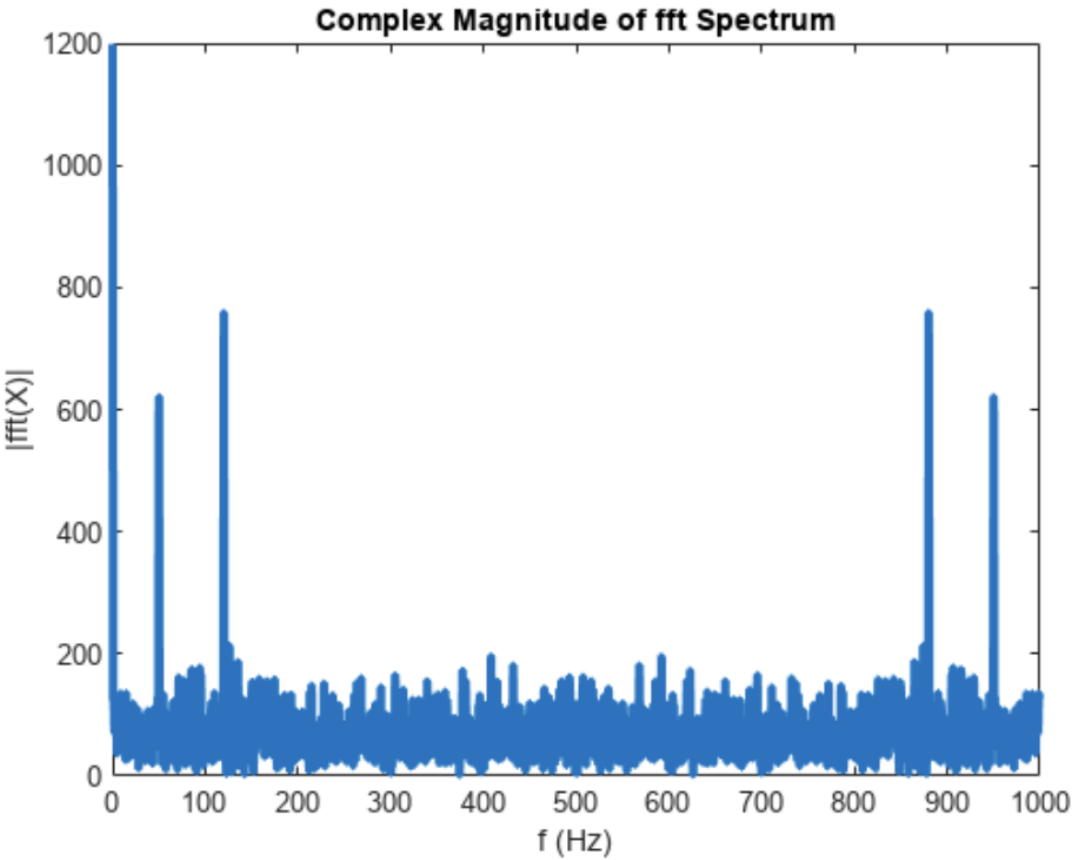
`Y = fft(X)`	N-point FFT, where N is the length of the input X
`Y = fft(X,n)`	n-point FFT, where n is specified as a function input
`Y = fft(X,n,dim)`	Multi-dimensional FFT (not relevant to this module)

- The output to this function is an N-point complex sequence
- To perform spectrum analysis, we are mainly interested in the amplitude of each component frequency in the signal, i.e., $|X[k]|$

MATLAB - FFT

An example fft spectrum:

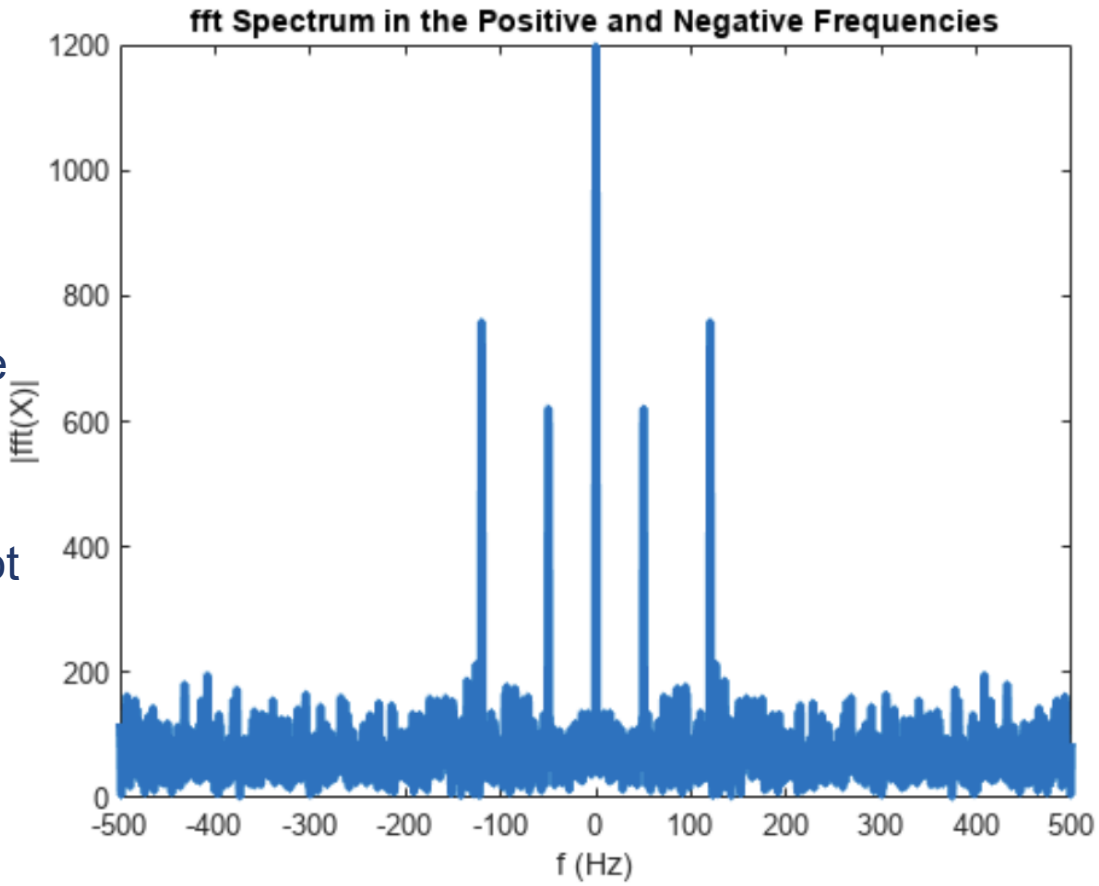
- 5 peaks in total
- 4 of the peaks are mirrored
- This is due to the symmetry of DFT
- The first half of its spectrum is in positive frequencies and the second half is in (aliased to) negative frequencies
- The first element reserved for the zero frequency (the highest peak in the figure)



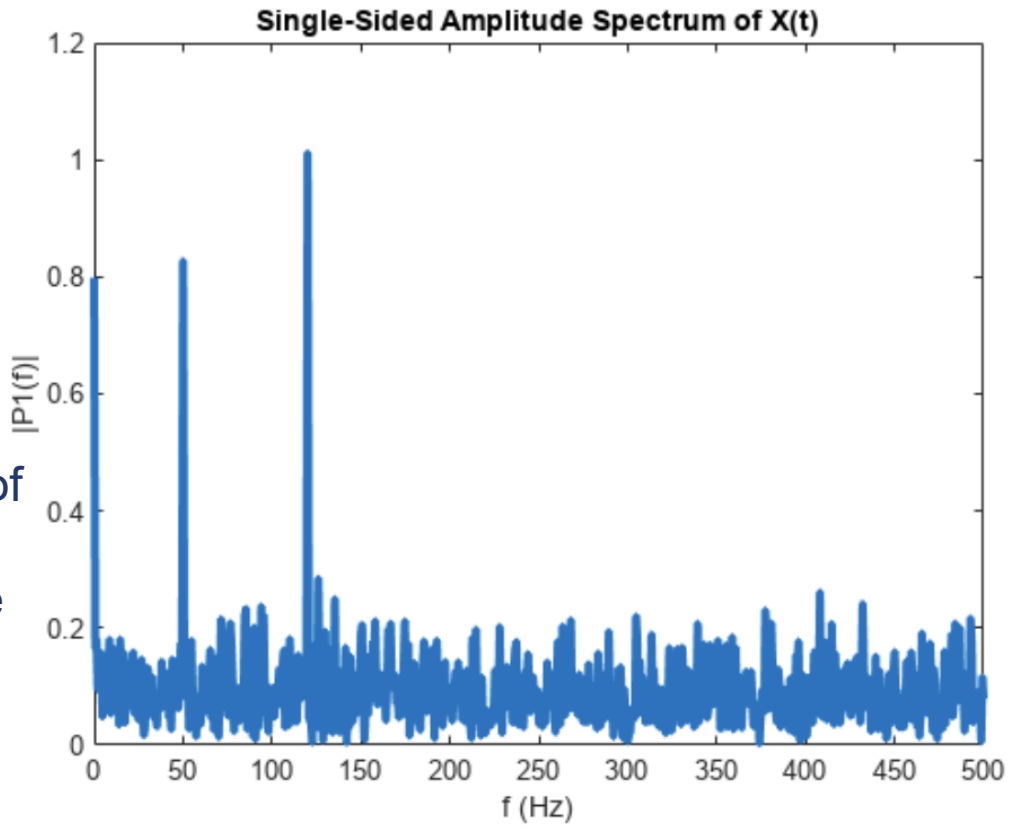
MATLAB - FFT

The two-sided fft spectrum:

- The same spectrum as in previous slide, but x-axis is shifted to reflect the +ve and -ve frequency values
- Now that we know the fft is always mirrored, we can just plot the spectrum of the positive frequencies
- This leads to the single-sided fft spectrum



MATLAB - FFT

The single-sided fft amplitude spectrum:

- We cannot simply just remove the negative frequencies
- To preserve the overall information, we also add up the fft magnitudes of the positive and negative frequencies (i.e., multiply amplitude by 2)
- Then, to obtain the amplitude spectrum, divide the fft magnitude by the number of samples